

## Table of Contents

|                                                                 |   |
|-----------------------------------------------------------------|---|
| M365 BEC IR: Native PowerShell Quick Reference .....            | 1 |
| Connect and disconnect.....                                     | 1 |
| Contain a compromised account (Graph).....                      | 2 |
| Inbox rules (Exchange Online).....                              | 2 |
| The three places forwarding hides.....                          | 3 |
| Delegation and permissions.....                                 | 3 |
| Enterprise apps and OAuth consent (Graph) .....                 | 3 |
| Entra ID risk (Graph) .....                                     | 4 |
| Unified Audit Log searches (Exchange Online) .....              | 4 |
| Message trace.....                                              | 5 |
| MFA methods, devices, and posture (Graph) .....                 | 6 |
| Remediation and hardening one-liners.....                       | 6 |
| Mass notification after a phishing blast.....                   | 7 |
| Download and run CrowdStrike CRT .....                          | 8 |
| Suspicious sign-in traits (correlate as many as possible) ..... | 8 |
| Consoles to review.....                                         | 8 |

## M365 BEC IR: Native PowerShell Quick Reference

Just the native M365 module commands. Use this when you do not have the scripts to hand, only the Exchange Online and Microsoft Graph modules.

Commands are provided as-is. Use at your own risk. None of this is legal advice.  
For legal advice, talk to a lawyer.

Modules used: ExchangeOnlineManagement, Microsoft.Graph, Microsoft.Graph.Beta.  
PowerShell 7.x recommended.

---

### Connect and disconnect

*# Connect IPPS first (so UAL check works), then EXO, then Graph (after EOL to avoid WAM conflict)*

Connect-IPPSession -EnableSearchOnlySession -DisableWAM

Connect-ExchangeOnline

Connect-MgGraph -Scopes

"AuditLog.Read.All", "AuditLogsQuery.Read.All", "Directory.Read.All", "Domain.Read.All", "User.Read.All", "Policy.Read.All", "Policy.Read.ConditionalAccess", "Id

```
entityRiskEvent.Read.All", "IdentityRiskyUser.Read.All", "Application.Read.All",  
"SecurityEvents.Read.All", "UserAuthenticationMethod.Read.All"
```

To contain/reset you need write scopes as well: add User.ReadWrite.All,  
Directory.ReadWrite.All, Policy.ReadWrite.ConditionalAccess,  
DelegatedPermissionGrant.ReadWrite.All, AppRoleAssignment.ReadWrite.All.

*# Disconnect when done*

```
Disconnect-ExchangeOnline -Confirm:$false ; Disconnect-MgGraph
```

---

## Contain a compromised account (Graph)

*# Block sign-in, reset to a random password, revoke sessions*

```
$upn = "user@contoso.com"
```

```
Update-MgUser -UserId $upn -AccountEnabled:$false
```

```
$pw = @{ Password =
```

```
([System.Web.Security.Membership]::GeneratePassword(20,4));
```

```
ForceChangePasswordNextSignIn = $true }
```

```
Update-MgUser -UserId $upn -PasswordProfile $pw
```

```
Revoke-MgUserSignInSession -UserId $upn
```

Re-enable later with `Update-MgUser -UserId $upn -AccountEnabled:$true`.

**AD-sync gotcha:** if the account is AD-synced without password writeback, the block and reset revert on the next sync. Change the password in on-prem AD and force a sync instead.

*# Revoke sessions for ALL users except the one you are signed in as*

```
$userlist = get-mguser -all | where {$_.UserPrincipalName -ne $(Get-MgContext).account} ;  
foreach ($user in $userlist) {revoke-mgusersigninsession -userid $user.id}
```

---

## Inbox rules (Exchange Online)

Inbox rules are the #1 BEC persistence/exfiltration mechanism: they survive password resets, are out of the user's sight, and they filter, forward, and delete evidence.

*# Review then remove a rule for one mailbox*

```
$mailbox = "user@contoso.com"
```

```
Get-Mailbox -Identity "$mailbox" | Select UserPrincipalName, Alias,  
DisplayName, Name, IsMailboxEnabled, SKUAssigned, IsDirSynced, WhenCreated,  
WhenChanged | FL
```

```
Get-InboxRule -IncludeHidden -Mailbox "$mailbox" | Select  
Name, Priority, Enabled, Description, Identity, From, SubjectContainsWords, SubjectO  
rBodyContainsWords, ForwardTo, MoveToFolder, SoftDeleteMessage, DeleteMessage |  
FL
```

```
Remove-InboxRule -Mailbox "$mailbox" -Identity "RULENAMEorID" -Confirm:$false
```

```
# All inbox rules across every mailbox (what script 19 does)
Get-Mailbox -ResultSize Unlimited | ForEach-Object { Get-InboxRule -
IncludeHidden -Mailbox $_.UserPrincipalName | Select
@{n="Mailbox";e={$_.MailboxOwnerId}},Name,Enabled,Description,ForwardTo,Forwa
rdAsAttachmentTo,RedirectTo,MoveToFolder,DeleteMessage }
```

**Suspicious-rule heuristics (script 17):** single-character or unicode-glyph rule names; external forward/redirect; moves mail to RSS Feeds / Conversation History / Deleted Items / Archive; finance keywords (invoice, wire, payroll, ACH, bank).

---

## The three places forwarding hides

```
# 1. Mailbox-level forwarding (script 20)
Get-Mailbox -ResultSize unlimited | Where-Object { ($_.ForwardingSMTPAddress
-ne $null) -or ($_.ForwardingAddress -ne $null) } | Select-Object Name,
UserPrincipalName, ForwardingSMTPAddress, ForwardingAddress,
DeliverToMailboxAndForward
```

```
# 2. Inbox-rule forwarding - see the inbox-rule commands above (ForwardTo /
RedirectTo / ForwardAsAttachmentTo)
```

```
# 3. Transport-rule forwarding (script 41)
```

```
Get-TransportRule -IncludeRecoverable | Where-Object { $_.RedirectMessageTo -
or $_.BlindCopyTo -or $_.CopyTo }
```

---

## Delegation and permissions

```
# Non-standard mailbox permissions (FullAccess) and SendAs
```

```
Get-Mailbox -ResultSize Unlimited | ForEach-Object { Get-MailboxPermission -
Identity $_.UserPrincipalName | Where-Object { $_.User -like "*@*" -and
$_.IsInherited -eq $false } }
Get-RecipientPermission -Identity "user@contoso.com" | Where-Object {
$_.Trustee -like "*@*" }
```

```
# FullAccess grants tenant-wide
```

```
Get-Mailbox -ResultSize Unlimited | Get-MailboxPermission | Where
{$_ .User.ToString() -ne "NT AUTHORITY\SELF" -and $_.IsInherited -eq $false} |
Select Identity,User,@{n='Rights';e=[string]::join(' ', $_.AccessRights)}}
```

---

## Enterprise apps and OAuth consent (Graph)

```
# List service principals (enterprise apps) and delegated consent grants
```

```
Get-MgServicePrincipal -All | Select DisplayName, AppId, Id,
AppOwnerOrganizationId, SignInAudience | Sort-Object AdditionalProperties
Get-MgOAuth2PermissionGrant -All | Select ClientId, ConsentType, PrincipalId,
ResourceId, Scope
```

```
# Inspect specific app's ('PerfectData Software') owners and role assignments
```

```
$sp = Get-MgServicePrincipal -Filter "DisplayName eq 'PerfectData Software'"
```

```
; Get-MgServicePrincipalOwner -ServicePrincipalId $sp.Id ; Get-  
MgServicePrincipalAppRoleAssignment -ServicePrincipalId $sp.Id
```

**Known-bad OAuth apps:** PerfectData Software and eM Client are frequently consented during BEC for mailbox exfiltration. Review any app consented inside the incident window.

---

## Entra ID risk (Graph)

*# Get risky users, risk detections*

```
Get-MgRiskyUser | Format-Table UserDisplayName, RiskLevel, RiskState,  
RiskLastUpdatedDateTime  
Get-MgRiskDetection | Format-Table UserDisplayName, RiskType, RiskLevel,  
DetectedDateTime, IpAddress
```

*# Only high-risk detections*

```
Get-MgRiskDetection -Filter "RiskLevel eq 'high'" | Format-Table  
UserDisplayName, RiskType, RiskLevel, DetectedDateTime, IpAddress
```

---

## Unified Audit Log searches (Exchange Online)

Search-UnifiedAuditLog returns max 5000 rows per call; page with -SessionId <guid> -SessionCommand ReturnLargeSet in a loop. Times are UTC. -StartDate, -EndDate, -UserIds, -IPAddresses, -RecordType, and -Operations are the filters that matter.

*# Sign-ins (script 12)*

```
Search-UnifiedAuditLog -StartDate (Get-Date).AddDays(-10) -EndDate (Get-Date)  
-RecordType azureactivedirectorystslogin -Operations  
"userloginfailed","userloggedin" -SessionCommand ReturnLargeSet -ResultSize  
5000
```

*# Inbox rule changes, even deleted rules (script 18)*

```
Search-UnifiedAuditLog -Operations New-InboxRule,Set-  
InboxRule,UpdateInboxRules,Remove-InboxRule,Disable-InboxRule -StartDate  
(Get-Date).AddDays(-10) -EndDate (Get-Date) -SessionCommand ReturnLargeSet -  
ResultSize 5000
```

*# High-value IR events (script 15)*

```
Search-UnifiedAuditLog -RecordType AzureActiveDirectory -Operations "Add  
member to role.,""Remove member from role." -StartDate $s -EndDate $e -  
SessionCommand ReturnLargeSet -ResultSize 5000  
Search-UnifiedAuditLog -RecordType AzureActiveDirectory -Operations "Change  
user password.,""Reset user password.,""Set force change user password." -  
StartDate $s -EndDate $e -SessionCommand ReturnLargeSet -ResultSize 5000  
Search-UnifiedAuditLog -Operations "Add-MailboxPermission","Remove-  
MailboxPermission","Add-RecipientPermission","Remove-  
RecipientPermission","Set-Mailbox" -StartDate $s -EndDate $e -SessionCommand  
ReturnLargeSet -ResultSize 5000
```

*# Consent to application (script 27)*

```
Search-UnifiedAuditLog -Operations "Consent to application." -StartDate (Get-Date).AddDays(-30) -EndDate (Get-Date) -SessionCommand ReturnLargeSet -ResultSize 5000
```

*# Activity by IP (36) and by user (37)*

```
Search-UnifiedAuditLog -IPAddresses "1.1.1.1","8.8.8.8" -StartDate $s -EndDate $e -SessionCommand ReturnLargeSet -ResultSize 5000
```

```
Search-UnifiedAuditLog -UserIds "compromised@contoso.com" -StartDate $s -EndDate $e -SessionCommand ReturnLargeSet -ResultSize 5000
```

*# What files were accessed (38) / what mail was accessed (39) / outbound blast (40)*

```
Search-UnifiedAuditLog -Operations FileAccessed -UserIds
```

```
"compromised@contoso.com" -StartDate $s -EndDate $e -SessionCommand ReturnLargeSet -ResultSize 5000
```

```
Search-UnifiedAuditLog -Operations "MailItemsAccessed","MessageBind" -UserIds "compromised@contoso.com" -StartDate $s -EndDate $e -SessionCommand ReturnLargeSet -ResultSize 5000
```

```
Search-UnifiedAuditLog -Operations Send,SendAs,SendOnBehalf -UserIds "compromised@contoso.com" -StartDate $s -EndDate $e -SessionCommand ReturnLargeSet -ResultSize 5000
```

*# Mailbox audit activity - Search-MailboxAuditLog is deprecated; use RecordType ExchangeItem (script 34)*

```
Search-UnifiedAuditLog -RecordType ExchangeItem -UserIds
```

```
"compromised@contoso.com" -StartDate $s -EndDate $e -SessionCommand ReturnLargeSet -ResultSize 5000
```

**MailItemsAccessed caveat:** it can be throttled ("too noisy to log"). Rows carry IsThrottled and MailAccessType=Sync flags in AuditData. When direct records are missing, infer access from dwell time, volume of other mailbox activity, forged messages, and OAuth grants.

---

## Message trace

Get-MessageTrace is being retired; use **Get-MessageTraceV2**.

*# Sent and received for one address*

```
$u = "user@contoso.com" ; $s = (Get-Date).AddDays(-10) ; $e = Get-Date
```

```
Get-MessageTraceV2 -SenderAddress $u -StartDate $s -EndDate $e -ResultSize 5000 | Export-Csv .\TraceSent.csv -NoTypeInfo
```

```
Get-MessageTraceV2 -RecipientAddress $u -StartDate $s -EndDate $e -ResultSize 5000 | Export-Csv .\TraceReceived.csv -NoTypeInfo
```

---

## MFA methods, devices, and posture (Graph)

```
# One user's registered auth methods and devices (script 31)
$upn = "user@contoso.com"
Get-MgUserAuthenticationMethod -UserId $upn | Format-List *
Get-MgUserRegisteredDevice -UserId (Get-MgUser -UserId $upn).Id

# Tenant posture: Security Defaults, CA policies, Secure Score (scripts
89/91/93)
Get-MgPolicyIdentitySecurityDefaultEnforcementPolicy | Format-List IsEnabled
Get-MgIdentityConditionalAccessPolicy -All | Select DisplayName, State
Get-MgBetaSecuritySecureScore -All | Select-Object -First 1 CurrentScore,
MaxScore, CreatedDateTime
```

---

## Remediation and hardening one-liners

```
# Block a sending IP - Connection Filter Policy (any license level)
Set-HostedConnectionFilterPolicy "Default" -IPBlockList
10.10.10.0/25,172.17.17.0/24
# Add/remove without disturbing existing entries:
Set-HostedConnectionFilterPolicy -Identity Default -IPBlockList
@{Add="192.168.3.0/24";Remove="192.168.1.10"}

# Block a sending IP - Tenant Allow/Block List (drops mail at the service
edge)
New-TenantAllowBlockListItems -ListType IP -Action Block -Entries
"2001:0db8:85a3:0000:0000:8a2e:0370:7334" -ExpirationDate (Get-
Date).AddDays(30) -Reason "Suspicious activity"

# Block executable attachments (transport rule)
New-TransportRule -Name "Block Executable Attachments" -
AttachmentHasExecutableContent $true -DeleteMessage $true

# External-sender warning banner (prepend)
$WarningBanner = '<table border=0 cellpadding=0 align="left"
width="100%"><tr><td style="background:#ffb900;padding:5pt 2pt 5pt
2pt"></td><td width="100%" style="background:#fff8e5;padding:5pt 4pt 5pt
12pt;word-wrap:break-word"><div style="color:#222222;"><span style="font-
weight:bold;">Warning: </span>This message originated outside of your email
system. Maintain caution when opening external links and
attachments.</div></td></tr></table><br/>'
New-TransportRule -Name "External message warning banner" -FromScope
NotInOrganization -SentToScope InOrganization -ApplyHtmlDisclaimerLocation
Prepend -ApplyHtmlDisclaimerText $WarningBanner -
ApplyHtmlDisclaimerFallbackAction Wrap

# Block display-name impersonation of internal users from external senders
$displayNames = (Get-EXOMailbox -ResultSize unlimited -RecipientTypeDetails
usermailbox).displayname
New-TransportRule -Name "Block impersonation attempt" -FromScope
```

```

NotInOrganization -SentToScope InOrganization -HeaderMatchesMessageHeader
From -HeaderMatchesPatterns $displayNames -Quarantine $true

# Disable server-side rule creation in OWA (stops users
viewing/creating/modifying server-side inbox rules)
Get-Mailbox -ResultSize unlimited | Set-CASMailbox -OWAMailboxPolicy
"Default"
Set-OwaMailboxPolicy -Identity OwaMailboxPolicy-Default -RulesEnabled $false

# List and unblock restricted senders (propagation can take up to 24h)
Get-BlockedSenderAddress | Format-Table DisplayName, EmailAddress, Reason,
BlockedDate
Remove-BlockedSenderAddress -SenderAddress "user@contoso.com"

# Verify and enable mailbox auditing
Get-OrganizationConfig | Format-List AuditDisabled
Set-OrganizationConfig -AuditDisabled $false
Set-Mailbox -Identity "user@contoso.com" -AuditOwner
@{Add="Create","HardDelete","MailboxLogin","MailItemsAccessed","Move","Search
QueryInitiated","Send","SoftDelete","Update","UpdateInboxRules"}
# Loop all user mailboxes:
$Mailboxes = Get-Mailbox -ResultSize Unlimited -Filter {RecipientTypeDetails
-eq "UserMailbox"} ; foreach ($Mailbox in $Mailboxes) { Set-Mailbox -Identity
$Mailbox.Identity -AuditOwner
@{Add="MailItemsAccessed","Send","UpdateInboxRules"} }

# If Get-OrganizationConfig shows IsDehydrated : True, run this before policy
edits
Enable-OrganizationCustomization

# Spam filter review
Get-HostedOutboundSpamFilterPolicy | Format-Table
Name,Enabled,IsDefault,Action*
Get-HostedContentFilterPolicy
Set-HostedContentFilterPolicy -Identity Default -HighConfidenceSpamAction
Quarantine

# Check security-defaults / auth policy (including legacy auth)
Get-MgPolicyIdentitySecurityDefaultEnforcementPolicy | FL
Get-AuthenticationPolicy

```

---

## Mass notification after a phishing blast

Build a recipient list from message trace, save the addresses to a text file (one per line), then:

```

Connect-ExchangeOnline
$AddressList = "Path\to\address\list.txt"
$GroupID = "PhishingWarning@contoso.com"
New-DistributionGroup -Name $GroupID -PrimarySmtpAddress

```



```
"return@emailaddress.com"
$ListMembers = Get-DistributionGroupMember -Identity $GroupID -ResultSize
Unlimited | Select -Expand PrimarySmtpAddress
Import-CSV $AddressList -Header "UPN" | ForEach { If ($ListMembers -contains
$_UPN) { Write-Output "$($_UPN) already a member" } Else { Add-
DistributionGroupMember -Identity $GroupID -Member $_UPN; Write-Output
"$($_UPN) added" } }
# Send the notice from a suitable account, then clean up:
Remove-DistributionGroup -Identity $GroupID
```

Example wording: <https://blog.hubspot.com/marketing/sample-letter-for-hacked-email>

---

## Download and run CrowdStrike CRT

Collects federation config/trust, client-access settings, remote-domain and SMTP forwarding, transport rules, FullAccess/SendAs/SendOnBehalf delegates, EXO PowerShell-enabled users, audit-bypass users, GAL-hidden mailboxes, and admin audit logging config.

### Invoke-WebRequest

```
"https://github.com/CrowdStrike/CRT/raw/refs/heads/main/Get-CRTReport.ps1" -
OutFile .\Get-CRTReport.ps1 ; .\Get-CRTReport.ps1 -WorkingDirectory
"$($env:userprofile)\Desktop\Investigation\CRTReport" -Interactive
```

---

## Suspicious sign-in traits (correlate as many as possible)

History deviation from the 30-day baseline; geolocation with poor reputation or impossible travel; cloud-hosting (DCH) or VPN ASNs, or known-bad IPs; unregistered devices; uncommon OS (Mac OS, Linux) for the user; automated user agents (axios, python-httpx); sign-ins that did not pass MFA, including "satisfied by claim in token" (token replay).

## Consoles to review

Tenant Allow/Block List (<https://security.microsoft.com/tenantAllowBlockList>), Threat Policies (<https://security.microsoft.com/threatpolicy>), Anti-spam/Connection filter (<https://security.microsoft.com/antispam>), Entra ID User Settings ([https://portal.azure.com/#view/Microsoft\\_AAD\\_IAM/ActiveDirectoryMenuBlade/~/\\_/UserSettings](https://portal.azure.com/#view/Microsoft_AAD_IAM/ActiveDirectoryMenuBlade/~/_/UserSettings)), SharePoint sharing (<https://go.microsoft.com/fwlink/?linkid=2185222>).

---

\*Companion to the DEF CON 34 workshop "Investigating and Responding to M365 Account Compromise on a Shoestring." Verify cmdlet syntax against current Microsoft documentation